PROGRAMACIÓN I · UNIDAD 05

Unidad 5. Arreglos y cadenas de caracteres: conservar, recorrer y buscar información

Programación I — 2026
1.º Bachillerato Tecnológico en Informática · DGETP / UTU

¿Qué aprenderemos en esta unidad?

En la Unidad 4 aprendimos a repetir instrucciones. Utilizamos ciclos para leer varias calificaciones, sumar sus valores, contar cuántas eran aprobatorias y calcular un promedio. Sin embargo, nuestro último programa tenía una limitación: la variable `nota` recibía un valor nuevo en cada iteración. Al finalizar, conocíamos la suma, el promedio y la cantidad de aprobados, pero **no conservábamos cada nota individual**.

Imaginemos que, después de obtener el promedio, el docente pregunta: «¿Cuál fue la tercera calificación?». O que necesita cambiar una nota que se introdujo equivocadamente. O que quiere encontrar la calificación más alta. ¿Podemos hacerlo con los datos que realmente guardamos?

En esta unidad resolveremos ese problema mediante **arreglos** (*arrays*). Aprenderemos a guardar varios valores del mismo tipo en posiciones identificables, recorrerlos mediante ciclos, consultar y modificar elementos, buscar información y obtener resultados. Después estudiaremos las **cadenas de caracteres** (`String`), que nos permiten trabajar con textos y reconocer algunas semejanzas y diferencias importantes respecto de los arreglos.

Aprenderemos a:

- Identificar cuándo una variable individual deja de ser suficiente.
- Declarar, crear e inicializar arreglos de tamaño fijo.
- Comprender índices, posiciones y la propiedad `length`.
- Leer, consultar, modificar y recorrer arreglos mediante `for`.
- Diferenciar la capacidad del arreglo de la cantidad de elementos efectivamente utilizados.
- Sumar, contar, calcular promedios, buscar y encontrar máximos o mínimos.
- Realizar búsquedas lineales y explicar cuándo terminan.
- Detectar errores de límites y trazarlos antes de ejecutar.
- Reconocer que `String` representa texto y utilizar métodos básicos como `length()`, `charAt()`, `equals()`, `substring()` e `indexOf()`.
- Resolver un pequeño registro de datos con pseudocódigo, Java y casos de prueba.

La meta no es memorizar cómo se escriben los corchetes. Queremos comprender qué información se conserva, cómo se localiza y qué hacemos con ella después. Seguiremos el recorrido: **problema → predicción → pseudocódigo → Java → trazado → pruebas → explicación**.

Cómo utilizar este capítulo de forma autónoma. Cuando encuentres una pregunta de predicción, escribe o piensa tu respuesta *antes* de revelar la explicación. Si te bloqueas, identifica primero qué datos tienes y qué resultado necesitas; consulta una pista y vuelve a intentarlo. Si ya entendiste el ejemplo, compruébalo con otros datos y explica qué cambiaría. En los ejercicios con soluciones, la página debe mostrar las pistas

antes de la resolución completa. Si estudias el Markdown directamente, oculta el párrafo de solución mientras realizas la predicción.

---

1. Una limitación de las variables que ya conocemos

1.1. Procesar no significa conservar

Recordemos una parte de nuestro programa anterior:

Java

```
int suma = 0;

for (int i = 1; i <= 5; i++) {
    System.out.print("Nota " + i + ": ");
    int nota = teclado.nextInt();
    suma += nota;
}
```

Este fragmento supone que ya existe un `Scanner` llamado `teclado`. Durante cada iteración se lee una nota y se incorpora a la suma. Si el usuario introduce `7, 5, 8, 6, 9`, obtenemos 35. Pero la variable local `nota` de cada iteración deja de estar disponible cuando termina esa iteración. Tampoco estamos guardando las cinco calificaciones en otras variables.

La suma es información nueva construida a partir de los datos; **no contiene una copia consultable de cada uno**.

1.2. ¿Y si declaramos cinco variables?

Experimento: ¿dónde quedó la tercera nota?

Fragmento para analizar:

Fragmento Java

```
int nota = 7;
nota = 5;
nota = 8;
nota = 6;
nota = 9;
System.out.println(nota);
```

Antes de ejecutarlo, predice la salida:

es **9**. Cada asignación reemplaza el valor anterior; la variable no conserva un historial. **¿En qué lugar quedó guardada la tercera nota, 8?** En ninguno que podamos consultar con este fragmento.

Podemos intentar una solución con cinco variables independientes (la que veremos enseguida). Así recuperaremos `nota3`, pero aparecerá otra limitación: los nombres `nota1`, `nota2`, `nota3` no se convierten por sí solos en una secuencia que un ciclo pueda recorrer. **Necesitamos datos identificados por posiciones, no por una lista cada vez mayor de nombres.**

Podríamos escribir:

Java

```
int nota1 = 7;
int nota2 = 5;
int nota3 = 8;
int nota4 = 6;
int nota5 = 9;
```

Ahora sí conocemos la tercera nota: `nota3`. Pero ¿qué pasaría con treinta estudiantes? ¿Deberíamos crear `nota1` hasta `nota30`? ¿Cómo construiríamos un ciclo que consultara esos nombres? Java no transforma automáticamente el número de un contador en una parte del nombre de una variable.

Necesitamos **un solo nombre para identificar un conjunto ordenado de valores**, y una manera de indicar cuál queremos consultar.

1.3. Una posible solución conceptual

Imaginemos cinco casilleros numerados:

Casillero	0	1	2	3	4
Nota guardada	7	5	8	6	9

Ahora podemos decir «consulta el casillero 2» y obtener la tercera nota, 8. También podemos recorrer los casilleros en orden para sumar, buscar o modificar un valor.

Esa es la idea central de un **arreglo**. Los números de los casilleros se denominan **índices**. Observa que comenzamos en cero: será importante comprender por qué el tercer elemento tiene índice 2.

1.4. Preguntas antes de continuar

Sin escribir Java todavía, responde:

1. ¿Por qué la suma de cinco notas no permite recuperar la tercera?
2. ¿Qué ventaja tiene identificar las notas mediante posiciones?
3. Si el primer casillero es el 0, ¿cuál será el índice del quinto?
4. ¿Qué deberíamos conservar si luego necesitamos corregir una nota?

---

2. ¿Qué es un arreglo en Java?

Un **arreglo** es una estructura que contiene una cantidad fija de elementos de un mismo tipo. Cada elemento puede consultarse o modificarse mediante un índice.

Un arreglo de enteros puede almacenar varias calificaciones; uno de `double`, varias temperaturas; uno de `String`, varios nombres. La declaración del tipo determina qué elementos pueden almacenarse.

2.1. Nuestro primer arreglo

Java

```
int[] notas = {7, 5, 8, 6, 9};
```

Esta instrucción declara una variable llamada `notas` e inicializa un arreglo de cinco enteros. Es un **fragmento** para estudiar la estructura, no un programa completo independiente.

Las llaves contienen los valores iniciales. Java los ubica en el orden indicado.

Índice	`notas[0]`	`notas[1]`	`notas[2]`	`notas[3]`	`notas[4]`
Valor	7	5	8	6	9

2.2. El nombre del arreglo y sus elementos

`notas` identifica el arreglo. Para seleccionar un elemento utilizamos corchetes:

Java

```
System.out.println(notas[2]);
```

La salida es `8`, porque el índice 2 identifica el **tercer** elemento.

No escribimos `nota2` ni una variable diferente por cada posición: utilizamos el nombre común y el índice.

2.3. Los índices comienzan en cero

Tres preguntas diferentes sobre una misma nota

Con `int[] notas = {7, 5, 8, 6, 9};`, distingue:

Pregunta	Respuesta
¿Cuál es la **tercera** nota?	8

Pregunta	Respuesta
¿Qué **índice** tiene la tercera nota?	2
¿Qué **valor** tiene `notas[3]`?	6, que es la cuarta nota

Comprueba sin ejecutar: ¿qué valor da `notas[0]`? ¿Cuál es el índice de la quinta nota? ¿En qué posición ordinal aparece `notas[1]`? Ahora escribe una instrucción que consulte la cuarta nota y otra que modifique su valor a 10. En la segunda necesitarás una asignación, no solo una consulta.

En Java, el primer elemento de un arreglo se encuentra en el índice 0.

- Primer elemento → índice 0.
- Segundo elemento → índice 1.
- Tercer elemento → índice 2.
- Quinto elemento → índice 4.

En un arreglo de cinco elementos, el último índice válido es 4. La cantidad de elementos y el último índice **no son el mismo número**.

Podemos expresarlo mediante una regla:

Último índice válido = cantidad de elementos − 1.

2.4. Recurso: índices y valores

> Para notas = {7, 5, 8, 6, 9}, el tercer elemento tiene índice 2 y valor 8. length es 5; los índices -1 y 5 producirían un error de límites.
>
> Predice el resultado y contrástalo con los ejemplos del capítulo.

Antes de continuar, localiza el primer, tercer y último elemento. Para cada uno, indica **posición ordinal**, **índice** y **valor**.

2.5. Arreglo no significa colección ilimitada

El arreglo creado mediante `{7, 5, 8, 6, 9}` tiene cinco elementos. No podemos agregarle una sexta posición mediante `notas[5]`.

Podemos cambiar valores de las posiciones existentes. Si necesitamos otra capacidad, debemos crear otro arreglo con un tamaño diferente y, si corresponde, copiar la información. Más adelante conoceremos otras estructuras de datos que permiten tamaños variables.

---

3. Declarar, crear e inicializar arreglos

Ya conocemos una forma de inicializar un arreglo directamente:

Java

```
int[] notas = {7, 5, 8, 6, 9};
```

Pero también necesitamos crear un arreglo antes de conocer los datos que recibirá.

3.1. Crear espacio para una cantidad fija

Java

```
int[] notas = new int[5];
```

Esta instrucción crea un arreglo de cinco enteros.

- `int[]` indica el tipo de arreglo.
- `notas` es el nombre de la variable.
- `new int[5]` crea el arreglo con capacidad para cinco elementos.

El tamaño se establece al crear el arreglo y **permanece fijo para ese objeto**.

3.2. ¿Qué contiene un arreglo recién creado?

Predicción antes de asignar datos

Fragmento para analizar:

Fragmento Java

```
int[] notas = new int[5];
System.out.println(notas[2]);
```

¿Qué mostrará?

0. No lo introdujo ningún estudiante: es el valor predeterminado de un elemento `int` recién creado. No debemos interpretar automáticamente cada cero como una nota registrada. El arreglo tiene **cinco posiciones disponibles**, pero todavía no hemos cargado cinco calificaciones.

En este momento basta con comprender el caso de `int`. Los valores iniciales de otros tipos aparecen más abajo como referencia, pero no son un requisito nuevo para comenzar a recorrer arreglos.

En Java, cuando creamos un arreglo mediante `new`, sus elementos reciben valores predeterminados según el tipo.

Por ejemplo, en `new int[5]` los cinco elementos comienzan valiendo cero. En `new double[5]`, comienzan con `0.0`; en un arreglo de `boolean`, con `false`; y en un arreglo de referencias como `String[]`, los elementos comienzan como `null`.

Esto no significa que hayamos ingresado cinco notas de cero. Solamente significa que el arreglo dispone de cinco posiciones y que Java les asignó valores iniciales predeterminados.

3.3. Asignar elementos individualmente

Java

```
int[] notas = new int[5];

notas[0] = 7;
notas[1] = 5;
notas[2] = 8;
notas[3] = 6;
notas[4] = 9;
```

Después de estas asignaciones, el contenido coincide con el arreglo del apartado anterior.

3.4. Declarar no es lo mismo que crear

Podemos declarar:

Java

```
int[] notas;
```

Pero una variable local declarada de este modo todavía no referencia un arreglo creado. No podemos utilizar `notas[0]` antes de asignarle uno.

Después podemos crear:

Java

```
notas = new int[5];
```

La comparación recupera un aprendizaje de la Unidad 2: declarar una variable y asignarle un valor son acciones diferentes.

3.5. Un primer programa completo

Java

```
public class Main {

    public static void main(String[] args) {

        int[] notas = {7, 5, 8, 6, 9};

        System.out.println("Primera nota: " + notas[0]);
        System.out.println("Tercera nota: " + notas[2]);
        System.out.println("Última nota: " + notas[4]);
    }
}
```

Salida esperada:

Salida esperada

```
Primera nota: 7
Tercera nota: 8
Última nota: 9
```

Prueba a cambiar el valor de `notas[2]` en la inicialización. ¿Qué otras salidas cambian? ¿Por qué?

---

4. Consultar, modificar y distinguir la longitud

4.1. Consultar no modifica

Una copia no mantiene una relación automática

Observa las instrucciones completas, ejecutadas en orden:

Java

```
int[] notas = {7, 5, 8, 6, 9};
int tercera = notas[2];
notas[2] = 10;
System.out.println(tercera);
System.out.println(notas[2]);
```

La salida es **8** y luego **10**. Cuando se asignó `tercera`, recibió el valor entero 8. Cambiar posteriormente una posición del arreglo no vuelve a ejecutar esa asignación.

Momento	`tercera`	`notas[2]`
Después de la copia	8	8

Momento	`tercera`	`notas[2]`
Después de cambiar el arreglo	8	10

Este comportamiento conecta con las asignaciones de la Unidad 2. Más adelante compararemos esta copia de un **elemento `int`** con la asignación de una **referencia al arreglo entero**.

Java

```
int[] notas = {7, 5, 8, 6, 9};
int tercera = notas[2];
```

Ahora `tercera` vale 8. Consultar la posición no modifica el arreglo.

Si después escribimos:

Java

```
notas[2] = 10;
```

la tercera posición pasa a valer 10, pero la variable `tercera` continúa valiendo 8. La lectura anterior guardó una copia de ese valor entero, **no una relación automática**.

4.2. Corregir una calificación

Supongamos que la tercera nota se registró como 8, pero era 9.

Java

```
notas[2] = 9;
```

No necesitamos modificar las otras cuatro posiciones.

Esta posibilidad responde directamente al problema con el que comenzamos: podemos localizar un elemento y corregirlo.

4.3. La propiedad length

Java permite conocer la cantidad de elementos de un arreglo mediante:

Java

```
notas.length
```

Si tenemos cinco notas, su resultado es 5.

Observa que `length` es una **propiedad del arreglo** y se escribe sin paréntesis. Más adelante estudiaremos `length()` para cadenas, que se escribe de otra manera.

4.4. Último índice válido

Si `notas.length` es 5, podemos utilizar los índices:

`0, 1, 2, 3, 4`

Por tanto, el último elemento se consulta con:

Java

```
notas[notas.length - 1]
```

No se consulta con `notas[notas.length]`.

4.5. ¿Qué sucede con un índice inexistente?

Considera:

Java

```
int[] notas = {7, 5, 8, 6, 9};
System.out.println(notas[5]);
```

El arreglo solamente tiene posiciones de 0 a 4. Intentar acceder al índice 5 provoca una excepción durante la ejecución (`ArrayIndexOutOfBoundsException`). No es un valor que Java rellene automáticamente ni significa «el último elemento».

También sería incorrecto acceder a `notas[-1]`.

4.6. Programa completo de modificación

Java

```
public class Main {

    public static void main(String[] args) {

        int[] notas = {7, 5, 8, 6, 9};

        System.out.println("Antes: " + notas[2]);

        notas[2] = 10;

        System.out.println("Después: " + notas[2]);
        System.out.println("Cantidad de elementos: " + notas.length);
```

```
    }
}
```

Salida:

Texto

```
Antes: 8
Después: 10
Cantidad de elementos: 5
```

Pregunta de control: ¿cambió la cantidad de elementos cuando modificamos la tercera nota? No: cambió su valor, no el tamaño del arreglo.

---

5. Recorrer un arreglo mediante for

Ya sabemos guardar datos. Pero si tenemos treinta calificaciones, no sería práctico escribir treinta instrucciones `println` ni treinta sumas.

Aquí encontramos la relación entre la Unidad 4 y la actual: **el arreglo conserva los datos; el ciclo permite procesarlos**.

5.1. Mostrar todas las notas

Construir el recorrido antes de escribirlo

Partimos de `int[] notas = {7, 5, 8, 6, 9};`. Si queremos mostrarlas todas, primero escribimos los **índices** que deben visitarse: `0, 1, 2, 3, 4`.

- ¿Desde dónde comienza el índice? Desde 0.
- ¿Cuál es la primera posición inexistente? La 5, que coincide con `notas.length`.
- ¿Qué condición permite visitar 0–4 sin llegar a 5? `i < notas.length`.
- ¿Cómo avanzamos? `i++`.
- ¿Qué hacemos en cada paso? Consultar `notas[i]`.

Ahora la escritura de `for (int i = 0; i < notas.length; i++)` deja de ser una receta: representa el recorrido que acabamos de construir.

Desafío de transferencia: describe los índices para recorrer el arreglo **desde el último hasta el primero**, y luego solamente los índices pares (`0, 2, 4`). Escribe los encabezados correspondientes antes de programar sus cuerpos.

Java

```
int[] notas = {7, 5, 8, 6, 9};

for (int i = 0; i < notas.length; i++) {
    System.out.println(notas[i]);
}
```

El contador `i` comienza en 0 porque esa es la primera posición. El ciclo continúa mientras `i` sea menor que la cantidad de elementos. Después de cada iteración, aumenta en uno.

La salida será:

Texto

```
7
5
8
6
9
```

5.2. ¿Por qué utilizamos i < notas.length?

Porque si `notas.length` es 5, el último índice válido es 4.

La condición:

Java

```
i < notas.length
```

permite `i = 0, 1, 2, 3, 4`, pero no permite entrar al cuerpo con `i = 5`.

Esta forma evita escribir un número fijo cuando el recorrido debe cubrir el arreglo completo.

5.3. Trazado de recorrido

Comprobación	i	¿i < 5?	Elemento leído
1	0	Verdadero	`notas[0] = 7`
2	1	Verdadero	`notas[1] = 5`
3	2	Verdadero	`notas[2] = 8`
4	3	Verdadero	`notas[3] = 6`
5	4	Verdadero	`notas[4] = 9`

Comprobación	i	¿i < 5?	Elemento leído
6	5	Falso	No se accede al arreglo

Tenemos cinco iteraciones y una sexta evaluación falsa, igual que en los ciclos estudiados anteriormente.

5.4. Error frecuente: <= length en este recorrido

La expresión `i <= notas.length` no es incorrecta en todos los programas. **Es incorrecta cuando el cuerpo intenta leer `notas[i]` en cada vuelta**, porque permite el índice `notas.length`, que no existe. Debemos justificar el límite por los índices que necesitamos consultar, no memorizar que `<=` está siempre prohibido.

Observa este encabezado:

Java

```
for (int i = 0; i <= notas.length; i++)
```

El contador llega a 5 y entra al cuerpo, porque `5 <= 5` es verdadero. Si allí intentamos consultar `notas[5]`, el programa falla por un índice inexistente.

La condición correcta para recorrer las cinco posiciones es:

Java

```
for (int i = 0; i < notas.length; i++)
```

La diferencia de un símbolo cambia el resultado del programa.

5.5. Simulador de recorrido

> Con notas = {7, 5, 8, 6, 9}, for recorre índices 0 a 4. Con i = 5, i < notas.length es falsa y no se accede a notas[5].
>
> Predice el resultado y contrástalo con los ejemplos del capítulo.

Antes de avanzar, responde: ¿qué índice se consultará?, ¿qué valor obtendremos?, ¿qué sucederá con `i` después del incremento?

5.6. Programa completo

Java

```
public class Main {

    public static void main(String[] args) {

        int[] notas = {7, 5, 8, 6, 9};

        for (int i = 0; i < notas.length; i++) {
            System.out.println("Posición " + i + ": " + notas[i]);
        }
    }
}
```

Prueba cambiar la cantidad de elementos de la inicialización. El ciclo seguirá utilizando `notas.length` y recorrerá el nuevo tamaño, sin modificar su condición.

---

6. Leer datos y almacenarlos en un arreglo

Hasta ahora escribimos las calificaciones dentro del código. Pero en un programa útil necesitaremos introducirlas durante la ejecución.

6.1. El algoritmo: leer y guardar

Necesitamos procesar cinco notas. Para cada posición debemos solicitar una calificación y almacenarla **en esa posición**.

Texto

```
INICIO
    CREAR notas con 5 posiciones

    PARA i DESDE 0 HASTA 4 HACER
        LEER notas[i]
    FIN PARA

    PARA i DESDE 0 HASTA 4 HACER
        MOSTRAR notas[i]
    FIN PARA
FIN
```

El primer ciclo guarda los datos. El segundo demuestra que después podemos consultarlos otra vez, sin volver a pedírselos al usuario.

6.2. Código Java completo

Java

```
import java.util.Scanner;

public class Main {

    public static void main(String[] args) {

        Scanner teclado = new Scanner(System.in);
        int[] notas = new int[5];

        // Primera etapa: guardar las notas.
        for (int i = 0; i < notas.length; i++) {
            System.out.print("Ingrese la nota " + (i + 1) + ": ");
            notas[i] = teclado.nextInt();
        }

        // Segunda etapa: volver a consultarlas.
        System.out.println("Notas registradas:");
        for (int i = 0; i < notas.length; i++) {
            System.out.println("Nota " + (i + 1) + ": " + notas[i]);
        }

        teclado.close();
    }
}
```

6.3. ¿Por qué mostramos i + 1?

Dentro del arreglo usamos índices 0, 1, 2, 3 y 4. Pero al hablar con una persona preferimos decir «nota 1», «nota 2» y así sucesivamente.

Por eso mostramos:

Java

```
(i + 1)
```

No estamos cambiando el índice del arreglo. Seguimos guardando en `notas[i]`. Solamente construimos un número de orden más familiar para el mensaje.

6.4. Ampliación: validar antes de guardar

Para el **aprendizaje esencial**, primero practica la lectura de cinco enteros válidos, guárdalos y muéstralos. Recién cuando ese recorrido funcione, incorpora la validación. Esta distinción evita aprender dos ciclos nuevos al mismo tiempo: uno para recorrer el arreglo y otro para repetir una entrada inválida.

Si las notas permitidas son de 1 a 10, podemos rechazar un valor fuera de rango. Durante esta primera práctica podemos leer una nota e informar si es inválida, pero si necesitamos completar exactamente cinco notas válidas conviene volver a solicitar esa posición.

Java

```
int nota;
do {
    System.out.print("Ingrese la nota " + (i + 1) + ": ");
    nota = teclado.nextInt();
    if (nota < 1 || nota > 10) {
        System.out.println("Nota inválida");
    }
} while (nota < 1 || nota > 10);

notas[i] = nota;
```

Este fragmento se coloca **dentro de un** `for` que controla `i`, y supone que ya existe el `Scanner` llamado `teclado`. Recupera el ciclo anidado de la Unidad 4. No es necesario dominarlo para comprender el arreglo, pero resulta útil como ampliación.

6.5. ¿Qué sucede después de cerrar la lectura?

El arreglo conserva los valores durante la ejecución mientras el programa dispone de él. Podemos recorrerlo varias veces sin volver a introducir las calificaciones.

Eso no significa que los datos queden guardados permanentemente en un archivo. Cuando termina este programa, no hemos creado ninguna forma de almacenamiento persistente.

---

7. Procesar datos guardados: suma, promedio y contadores

En la Unidad 4 aprendimos a calcular resultados mientras recibíamos datos. Ahora podemos hacerlo **después de guardarlos**, recorriendo el arreglo tantas veces como necesitemos.

7.1. Sumar las notas

Java

```
int[] notas = {7, 5, 8, 6, 9};
int suma = 0;

for (int i = 0; i < notas.length; i++) {
    suma = suma + notas[i];
}
```

Al finalizar, `suma` vale 35.

El acumulador se inicializa antes del ciclo y recibe el valor del elemento actual en cada iteración.

7.2. Contar aprobados

Java

```
int aprobados = 0;

for (int i = 0; i < notas.length; i++) {
    if (notas[i] >= 6) {
        aprobados++;
    }
}
```

El contador aumenta solamente cuando el elemento cumple la condición. Para estos datos, vale 4.

7.3. Calcular el promedio

Java

```
double promedio = suma * 1.0 / notas.length;
```

Utilizamos `1.0` para evitar la división entera, tal como aprendimos en la Unidad 2.

Esta operación supone que el arreglo contiene al menos un elemento. Si su longitud fuera cero, no tendríamos notas para promediar y deberíamos informar esa situación en vez de calcular un promedio.

7.4. Trazado de suma y aprobados

Índice	Nota	Suma antes	Suma después	Aprobados después
Inicial	—	—	0	0
0	7	0	7	1
1	5	7	12	1
2	8	12	20	2
3	6	20	26	3
4	9	26	35	4

La tabla se puede reconstruir sin ejecutar el programa. Esa es una prueba de comprensión del algoritmo.

7.5. Programa completo

Java

```
public class Main {

    public static void main(String[] args) {

        int[] notas = {7, 5, 8, 6, 9};
        int suma = 0;
        int aprobados = 0;

        for (int i = 0; i < notas.length; i++) {
            suma += notas[i];

            if (notas[i] >= 6) {
                aprobados++;
            }
        }

        double promedio = suma * 1.0 / notas.length;

        System.out.println("Suma: " + suma);
        System.out.println("Promedio: " + promedio);
        System.out.println("Aprobados: " + aprobados);
    }
}
```

Salida:

Texto

```
Suma: 35
Promedio: 7.0
Aprobados: 4
```

7.6. Lo que cambia respecto de la Unidad 4

Dos momentos distintos: sumar al leer o volver a calcular

En la Unidad 4, una nota nueva reemplazaba la anterior después de incorporarla a la suma. Aquí primero **conservamos** las notas y luego recorremos el arreglo. Esto nos permite hacer otra operación, buscar una nota o corregir un dato sin solicitar todo nuevamente.

Pero conservar los datos **no significa que los resultados se actualicen solos**:

Java

```
int[] notas = {7, 5, 8, 6, 9};
int suma = 35;
```

```
double promedio = suma / 5.0;
notas[2] = 10;
System.out.println(promedio);
```

¿Qué muestra?

7.0. Aunque ahora la tercera nota vale 10, el promedio quedó calculado con la suma anterior. Para obtener el promedio actualizado, debemos recorrer y volver a sumar los valores: `7 + 5 + 10 + 6 + 9 = 37`, y entonces `37 / 5.0 = 7.4`.

Actividad de predicción: ¿qué variable debes reinicializar antes de volver a recorrer el arreglo? ¿Por qué no basta con ejecutar otra vez una división usando `suma = 35`?

Antes procesábamos una nota y luego la reemplazábamos por la siguiente entrada. Ahora conservamos las cinco y podemos ejecutar varios recorridos posteriores sin volver a solicitarlas. **El acumulador guarda un resultado; el arreglo guarda los valores que permiten obtenerlo.**

---

8. Profundización: capacidad y cantidad de elementos utilizados

Hay una situación que merece atención. Imaginemos que creamos un arreglo con diez posiciones porque ese es el máximo permitido, pero solamente ingresamos cuatro notas.

Java

```
int[] notas = new int[10];
int cantidad = 4;
```

Aquí:

- `notas.length` vale 10: es la **capacidad** del arreglo.
- `cantidad` vale 4: representa cuántas posiciones contienen datos que decidimos registrar.

No debemos confundir ambos valores.

8.1. ¿Qué pasa si recorremos las diez posiciones?

Los elementos `int` que todavía no hemos asignado contienen el valor predeterminado 0. Si los tratamos como calificaciones reales, podríamos sumar o promediar datos que el usuario nunca introdujo.

Por eso, cuando utilizamos solamente parte del arreglo, podemos recorrer:

Java

```
for (int i = 0; i < cantidad; i++) {
    System.out.println(notas[i]);
```

```
}
```

8.2. Una regla de coherencia

Esta cantidad debe validarse siempre que provenga de una entrada o de otro cálculo:

Java

```
cantidad >= 0 && cantidad <= notas.length
```

El límite inferior evita cantidades negativas; el superior impide recorrer posiciones que no existen. Esta profundización es útil para registros con capacidad reservada, pero no es imprescindible para empezar a trabajar con arreglos cuyos cinco elementos ya están cargados.

Para un arreglo de capacidad diez, `cantidad` debe mantenerse entre 0 y 10 inclusive. Si intentamos registrar once elementos, ya no existe una posición donde guardar el undécimo.

Además, si `cantidad` vale 0, todavía no tenemos datos válidos para calcular un promedio.

8.3. Programa completo con cuatro datos en diez posiciones

Java

```
public class Main {

    public static void main(String[] args) {

        int[] notas = new int[10];
        int cantidad = 4;

        notas[0] = 7;
        notas[1] = 5;
        notas[2] = 8;
        notas[3] = 6;

        for (int i = 0; i < cantidad; i++) {
            System.out.println(notas[i]);
        }
    }
}
```

Salida: 7, 5, 8 y 6, una nota en cada línea. Las otras seis posiciones existen, pero no forman parte de los datos registrados.

8.4. Una pregunta de diseño

¿Qué conviene utilizar para controlar un recorrido: `notas.length` o `cantidad`?

Depende del problema. Si todas las posiciones representan datos, recorremos `length`. Si el arreglo tiene capacidad reservada pero solo una parte contiene datos registrados, recorremos `cantidad`.

La diferencia será especialmente importante al buscar y procesar registros simples.

---

9. Buscar un dato: búsqueda lineal

Guardar un conjunto de valores permite responder preguntas que antes no podíamos resolver cómodamente.

Imaginemos que necesitamos saber si alguna nota es igual a 8.

En un arreglo pequeño podemos mirar los elementos, pero si contiene muchos datos necesitamos un algoritmo.

9.1. Primera necesidad: ¿existe el valor?

Una **búsqueda lineal** revisa los elementos en orden hasta encontrar el valor buscado o llegar al final del conjunto utilizado.

Con el arreglo:

`7, 5, 8, 6, 9`

Y el objetivo 8:

- Consultamos índice 0: valor 7; no coincide.
- Consultamos índice 1: valor 5; no coincide.
- Consultamos índice 2: valor 8; coincide.

Ya sabemos que el valor existe y dónde aparece por primera vez.

9.2. Primera versión: recorrer todo y registrar si existe

Primera meta: obtener solamente una respuesta de sí o no. Todavía no necesitamos conservar posiciones ni detener anticipadamente el ciclo. Podemos comprobar el resultado con tres casos: buscar 7 (primero), 9 (último) y 4 (ausente). El valor de `encontrado` comienza en `false` y cambia a `true` cuando aparece al menos una coincidencia.

Java

```
int[] notas = {7, 5, 8, 6, 9};
int buscado = 8;
boolean encontrado = false;

for (int i = 0; i < notas.length; i++) {
    if (notas[i] == buscado) {
        encontrado = true;
```

```
    }
}
```

Esta versión recorre todas las posiciones. Cuando encuentra una coincidencia, cambia la bandera a `true`. Si no encuentra ninguna, permanece `false`.

Es adecuada para comprender la búsqueda sin introducir nuevas instrucciones de salida anticipada del ciclo.

9.3. Guardar el índice de la primera coincidencia

Segunda meta: informar **dónde** aparece por primera vez. `posicion` comienza en -1 porque ningún índice legal de Java es negativo. El `for` de esta versión **sigue recorriendo hasta el final**, pero la condición `posicion == -1` impide reemplazar el primer índice encontrado. Es decir, conservar la primera coincidencia **no es lo mismo** que detener la búsqueda en ese instante.

Con `{7, 5, 8, 8, 9}` y `buscado = 8`, la posición final será 2, no 3.

Ahora queremos conocer la posición.

Java

```
int posicion = -1;

for (int i = 0; i < notas.length; i++) {
    if (notas[i] == buscado && posicion == -1) {
        posicion = i;
    }
}
```

El valor -1 significa aquí «todavía no encontré ninguna posición». Es seguro utilizarlo como indicador porque ningún índice válido de Java es negativo.

La condición `posicion == -1` evita reemplazar la primera coincidencia si el valor aparece varias veces.

9.4. Buscar hasta encontrar: bandera y while

Tercera meta: detener el recorrido al encontrar la primera coincidencia. La pregunta nueva es: si el dato aparece en el índice 2, ¿para qué consultar los índices 3 y 4 cuando solamente nos interesa la primera aparición? Esta versión puede ahorrar comparaciones. Trabájala después de dominar la búsqueda de existencia y la conservación del primer índice.

También podemos evitar seguir recorriendo cuando ya encontramos el dato:

Java

```
int i = 0;
boolean encontrado = false;

while (i < notas.length && !encontrado) {
```

```
    if (notas[i] == buscado) {
        encontrado = true;
    } else {
        i++;
    }
}
```

Si termina con `encontrado == true`, `i` conserva el índice encontrado. Si termina sin coincidencia, `i` vale `notas.length`.

Observa la condición: `i < notas.length && !encontrado`. La primera parte comprueba el límite y la segunda consulta la bandera. **En este ejemplo**, invertir esas dos expresiones no produciría por sí solo un acceso inválido, porque ninguna de las dos consulta `notas[i]`. Sin embargo, el orden sí es imprescindible si la segunda parte accede al arreglo, como aquí:

Java

```
while (i < notas.length && notas[i] != buscado) {
    i++;
}
```

En esa variante, el cortocircuito de `&&` impide intentar `notas[i]` cuando `i` ya alcanzó la longitud. **No confundas la lógica de parada con la seguridad del acceso**.

9.5. Pseudocódigo de la búsqueda

Texto

```
INICIO
    notas ← [7, 5, 8, 6, 9]
    buscado ← 8
    indice ← 0
    encontrado ← FALSO

    MIENTRAS indice < 5 Y NO encontrado HACER
        SI notas[indice] = buscado ENTONCES
            encontrado ← VERDADERO
        SINO
            indice ← indice + 1
        FIN SI
    FIN MIENTRAS

    SI encontrado ENTONCES
        MOSTRAR indice
    SINO
        MOSTRAR "No encontrado"
    FIN SI
FIN
```

En pseudocódigo utilizamos `=` para expresar igualdad en una condición según nuestra convención; en Java escribimos `==`.

9.6. Simulador de búsqueda lineal

> Buscar 8 en {7, 5, 8, 6, 9}: se comparan índices 0, 1 y 2; la primera posición es 2. Buscar 4 recorre las cinco posiciones y comunica que no está.
>
> Predice el resultado y contrástalo con los ejemplos del capítulo.

Antes de avanzar, predice qué posición se comprobará y si el algoritmo continuará. Prueba un valor que está al comienzo, otro al final y uno que no está.

9.7. Programa completo

Java

```
public class Main {

    public static void main(String[] args) {

        int[] notas = {7, 5, 8, 6, 9};
        int buscado = 8;

        int i = 0;
        boolean encontrado = false;

        while (i < notas.length && !encontrado) {
            if (notas[i] == buscado) {
                encontrado = true;
            } else {
                i++;
            }
        }

        if (encontrado) {
            System.out.println("Encontrado en índice " + i);
        } else {
            System.out.println("No encontrado");
        }
    }
}
```

Prueba reemplazar `buscado` por 7, 9 y 4. Predice primero el resultado.

9.8. Qué demuestra una búsqueda

La búsqueda no necesita ordenar previamente los datos. Comprueba elementos uno por uno. En el peor caso debe examinar todos los elementos utilizados. Cuando aparecen valores repetidos, debemos definir si queremos encontrar cualquiera, el primero o todas las posiciones. La versión con `while` encuentra el primero.

---

10. Encontrar el mayor, el menor y contar coincidencias

Algunas preguntas no buscan un valor determinado: necesitan comparar los elementos entre sí.

Por ejemplo: «¿Cuál es la nota más alta?».

10.1. No comenzar el máximo en cero por costumbre

Un error que parece razonable

Antes de mirar la solución, prueba mentalmente:

Java

```
int[] numeros = {-4, -9, -2, -7};
int mayor = 0;
for (int i = 0; i < numeros.length; i++) {
    if (numeros[i] > mayor) {
        mayor = numeros[i];
    }
}
System.out.println(mayor);
```

¿Qué muestra?

0, aunque no aparece en el arreglo. Todos los números son menores que cero y el supuesto máximo no se actualiza. Así descubrimos por qué conviene comenzar con `mayor = numeros[0]` cuando el arreglo no está vacío. Luego podemos recorrer desde el índice 1: el primero ya fue considerado.

Para notas de 1 a 10, un máximo inicial de cero podría funcionar. Pero queremos construir una estrategia general que también sirva para arreglos con valores negativos.

Una solución es tomar como máximo inicial el primer elemento:

Java

```
int mayor = numeros[0];
```

Después recorremos desde el índice 1 y actualizamos cuando encontramos un valor superior.

Esta estrategia supone que el arreglo tiene al menos un elemento.

10.2. Buscar el mayor

Java

```
int[] numeros = {7, 5, 8, 6, 9};
int mayor = numeros[0];

for (int i = 1; i < numeros.length; i++) {
    if (numeros[i] > mayor) {
        mayor = numeros[i];
    }
}
```

Al finalizar, `mayor` vale 9.

10.3. Trazado de máximo

Índice	Valor	Mayor antes	¿Actualiza?	Mayor después
Inicial	7	—	Inicializar	7
1	5	7	No	7
2	8	7	Sí	8
3	6	8	No	8
4	9	8	Sí	9

10.4. Encontrar el menor

La lógica es similar:

Java

```
int menor = numeros[0];

for (int i = 1; i < numeros.length; i++) {
    if (numeros[i] < menor) {
        menor = numeros[i];
    }
}
```

Para los datos anteriores, el menor es 5.

10.5. Contar coincidencias no es buscar una sola

Imaginemos las notas:

`7, 8, 8, 6, 8`

Si queremos saber si existe un 8, alcanza con una búsqueda. Pero si queremos saber **cuántas veces** aparece, necesitamos recorrer todos los elementos y utilizar un contador.

Java

```
int[] notas = {7, 8, 8, 6, 8};
int cantidadOchos = 0;

for (int i = 0; i < notas.length; i++) {
    if (notas[i] == 8) {
        cantidadOchos++;
    }
}
```

El resultado es 3.

Aquí aparecen dos necesidades distintas: **existencia** y **cantidad de coincidencias**. Elegimos el algoritmo según lo que pide el problema.

10.6. Programa completo de máximo y mínimo

Java

```
public class Main {

    public static void main(String[] args) {

        int[] numeros = {-4, 12, 3, -8, 7};

        if (numeros.length > 0) {
            int mayor = numeros[0];
            int menor = numeros[0];

            for (int i = 1; i < numeros.length; i++) {
                if (numeros[i] > mayor) {
                    mayor = numeros[i];
                }
                if (numeros[i] < menor) {
                    menor = numeros[i];
                }
            }

            System.out.println("Mayor: " + mayor);
            System.out.println("Menor: " + menor);
        } else {
            System.out.println("No hay datos");
        }
```

```
    }
}
```

Salida:

Texto

```
Mayor: 12
Menor: -8
```

Este ejemplo permite comprobar que la inicialización con el primer elemento también sirve cuando existen números negativos.

---

11. Cadenas de caracteres: trabajar con texto mediante String

Hasta ahora trabajamos principalmente con arreglos numéricos. Pero los programas también necesitan procesar nombres, códigos, mensajes y otras informaciones textuales.

Ya utilizamos `String` para guardar texto:

Java

```
String nombre = "Ana María";
```

Ahora estudiaremos algunas operaciones que permiten consultar y trabajar con ese contenido.

11.1. ¿String es un arreglo de char?

Ruta de aprendizaje dentro de esta sección

- **Imprescindible:** `length()`, `charAt()` y recorrido mediante `for`.
- **Consolidación:** `equals()`, `equalsIgnoreCase()` e `indexOf()` para comparar y buscar.
- **Ampliación:** `substring()`, `toUpperCase()` y `toLowerCase()` para obtener cadenas nuevas.

No necesitas memorizar todos los métodos al mismo tiempo. Primero aprende a consultar índices y recorrer el texto como otra secuencia de datos, recordando que no se modifica del mismo modo que un arreglo.

No debemos tratar `String` como si fuera un `char[]`.

Una cadena representa una secuencia de caracteres y proporciona métodos para trabajar con ellos, pero `String` es una clase de Java. En particular, **las cadenas son inmutables**: no podemos modificar directamente el carácter de una posición como haríamos con un elemento de un arreglo.

Podemos consultar un carácter, comparar textos u obtener una nueva cadena a partir de otra.

11.2. La longitud de una cadena

Un mismo problema de consulta, dos sintaxis

Arreglo:

Java

```
int[] numeros = {3, 5, 8};
System.out.println(numeros[1]);  // Muestra 5
System.out.println(numeros.length);  // Muestra 3
```

Cadena:

Java

```
String palabra = "Java";
System.out.println(palabra.charAt(1));  // Muestra a
System.out.println(palabra.length());  // Muestra 4
```

Ambos permiten trabajar con posiciones numeradas desde cero. No obstante, **el arreglo usa `[]` para acceder al elemento y la cadena usa `charAt()`**; la longitud del arreglo es una propiedad y la de la cadena se obtiene mediante un método.

Java

```
String nombre = "Ana";
System.out.println(nombre.length());
```

Muestra `3`.

Observa la diferencia:

- Arreglo: `notas.length`.
- Cadena: `nombre.length()`.

En el primero utilizamos una propiedad. En el segundo, llamamos a un método.

11.3. Consultar un carácter con charAt

Caso límite: la cadena vacía

Java

```
String vacia = "";
System.out.println(vacia.length());  // Muestra 0
```

La cadena tiene longitud cero: no contiene un carácter en el índice 0. Intentar `vacia.charAt(0)` produce `StringIndexOutOfBoundsException`. Como con los arreglos, debemos comprobar que el índice esté dentro de los límites.

Java

```
String palabra = "Java";
System.out.println(palabra.charAt(0));
System.out.println(palabra.charAt(2));
```

Salida:

Texto

```
J
v
```

Al igual que con los arreglos, comenzamos a contar posiciones desde cero.

Para la palabra `Java`:

Índice	0	1	2	3
Carácter	J	a	v	a

El último índice válido para `charAt` es `palabra.length() - 1`, siempre que la cadena no esté vacía.

11.4. Recorrer una cadena

Java

```
String palabra = "Java";

for (int i = 0; i < palabra.length(); i++) {
    System.out.println(palabra.charAt(i));
}
```

La salida muestra cada carácter en una línea.

El ciclo es semejante al recorrido de un arreglo, pero la consulta se realiza mediante `charAt(i)` en lugar de corchetes.

11.5. Comparar contenidos con equals

En la Unidad 3 aprendimos que para comparar el contenido de cadenas utilizamos `equals()`:

Java

```
String respuesta = "si";

if (respuesta.equals("si")) {
    System.out.println("Respuesta afirmativa");
}
```

No debemos confiar en `==` para comparar contenido de cadenas, porque `String` es una clase y ese operador compara referencias.

Si queremos ignorar diferencias de mayúsculas y minúsculas, podemos utilizar `equalsIgnoreCase()`.

11.6. Buscar un fragmento con indexOf

Java

```
String mensaje = "Programacion Java";
int posicion = mensaje.indexOf("Java");
```

`posicion` vale 13, porque la `J` comienza en el índice 13. Si el fragmento buscado no aparece, `indexOf` devuelve -1.

Esto se parece conceptualmente a buscar un dato en un arreglo, aunque aquí utilizamos un método que Java ya proporciona.

11.7. Obtener una parte con substring

Java

```
String palabra = "Programar";
String parte = palabra.substring(0, 4);
System.out.println(parte);
```

La salida será:

Texto

```
Prog
```

En `substring(inicio, fin)`, el índice inicial está incluido y el final está excluido. Por eso `substring(0, 4)` obtiene los caracteres de índices 0, 1, 2 y 3.

11.8. Convertir mayúsculas y minúsculas

Java

```
String palabra = "Java";
String mayusculas = palabra.toUpperCase();
String minusculas = palabra.toLowerCase();
```

Ahora `mayusculas` contiene `JAVA` y `minusculas` contiene `java`. La variable `palabra` sigue conteniendo `Java`.

Los métodos construyen otros resultados de texto; no modifican la cadena original.

11.9. Un error frecuente: querer cambiar un carácter por índice

Con un arreglo de caracteres podemos modificar un elemento:

Java

```
char[] letras = {'J', 'a', 'v', 'a'};
letras[0] = 'L';
```

Pero no podemos escribir:

Java

```
String palabra = "Java";
palabra[0] = 'L';
```

Esto no es Java válido. `String` no utiliza ese mecanismo para modificar caracteres.

11.10. Recurso comparativo: arreglo y cadena

> Para numeros = {3, 5, 8} y palabra = "Java", numeros.length es 3 y palabra.length() es 4. numeros[1] es 5 y palabra.charAt(1) es a. Un elemento del arreglo puede cambiar; String es inmutable y palabra[0] = 'L' no compila.
>
> Predice el resultado y contrástalo con los ejemplos del capítulo.

11.11. Un programa completo: contar letras a

Java

```
public class Main {

    public static void main(String[] args) {

        String palabra = "banana";
        int cantidad = 0;

        for (int i = 0; i < palabra.length(); i++) {
            if (palabra.charAt(i) == 'a') {
                cantidad++;
            }
        }

        System.out.println("Cantidad de letras a: " + cantidad);
    }
}
```

La salida será `Cantidad de letras a: 3`.

La comparación utiliza `==` entre valores `char`, no entre objetos `String`. Esto muestra por qué debemos atender al tipo de dato que realmente estamos comparando.

11.12. Nota opcional: caracteres Unicode

En ejemplos con letras simples de nuestro alfabeto, recorrer mediante `charAt` resulta apropiado para aprender. Sin embargo, algunos símbolos Unicode se representan mediante más de una unidad `char`. Por eso `length()` y `charAt()` no equivalen en todos los casos a contar y obtener caracteres visuales completos. No necesitamos estudiar todavía esos detalles para los ejercicios introductorios.

11.13. Transferencia: decidir qué necesitamos conservar

Problema nuevo. Un docente quiere introducir las notas de diez estudiantes. Al finalizar necesita calcular el promedio, corregir una nota que se introdujo mal y volver a consultar la tercera. Antes de escribir Java, responde: ¿bastan `suma`, `contador` y `notaActual`?, ¿qué información perderíamos?, ¿qué estructura necesita el programa?

Pista 1. Diferencia *resultado agregado* de *dato individual recuperable*. **Pista 2.** Escribe qué valor consulta `notas[2]` si el arreglo comienza en índice 0. Piensa cómo realizar la corrección sin volver a pedir las diez notas.

Diseño que debes contrastar al terminar: un arreglo de diez enteros conserva cada dato; un ciclo `for` sirve para leerlo y otro para procesarlo. El promedio calculado no se actualiza automáticamente cuando cambia un elemento: vuelve a sumar o actualiza conscientemente el resultado conforme al requisito.

Depuración de búsqueda. Considera `{5, 8, 5, 3}`. Si necesitamos la primera posición del 5, el resultado es índice 0. Si necesitamos contar coincidencias, el resultado es 2. ¿Qué ocurre si guardamos la posición cada vez que encontramos un 5 y no detenemos ni protegemos la primera coincidencia? Terminaríamos guardando la última posición coincidente. ¿Qué resultado esperas cuando buscamos 9? Explica cómo representa el algoritmo la ausencia.

Transferencia de cadenas: decide qué cambia cuando buscamos un carácter dentro de un `String`: la longitud se obtiene mediante `length()`, un carácter mediante `charAt(i)` y no es posible asignar `texto[i] = ...` como en un arreglo. Distingue consulta de modificación.

Nota técnica sobre Scanner. En estos ejemplos `teclado.close()` se utiliza al terminar `main`. Un `Scanner` creado a partir de `System.in` cierra también la entrada estándar al cerrarse. Si más adelante llamas a varios métodos que leen datos, comparte un `Scanner` y evita cerrarlo antes de finalizar todas las lecturas.

---

12. Problema integrador: registro de calificaciones consultable

Vamos a construir un programa pequeño que utilice un arreglo para guardar calificaciones y permita consultarlas después.

No necesitamos bases de datos, archivos ni interfaces gráficas. Utilizaremos `Main.java`, `Scanner`, arreglos, ciclos y condiciones.

12.1. El problema

Un docente necesita registrar cinco calificaciones enteras entre 1 y 10. El programa debe:

1. Solicitar cada nota y almacenarla en un arreglo.
2. Mostrar las notas guardadas.
3. Calcular el promedio.
4. Contar cuántas notas son aprobatorias (6 o más).
5. Permitir consultar una nota mediante su número de orden, del 1 al 5.

El programa procesará una consulta y finalizará. No necesitamos todavía construir un menú complejo repetitivo.

12.2. Versión A: aprendizaje esencial

La primera versión supone que se introducen notas válidas. El objetivo es comprobar que el estudiante domina:

- Creación de un arreglo fijo.
- Lectura por índice.
- Recorrido mediante `for`.
- Acumulación y conteo.
- Consulta de una posición y traducción entre orden humano e índice Java.

Pseudocódigo

Texto

```
INICIO
    CREAR notas con 5 posiciones
    suma ← 0
    aprobados ← 0

    PARA i DESDE 0 HASTA 4 HACER
        LEER notas[i]
    FIN PARA

    PARA i DESDE 0 HASTA 4 HACER
        MOSTRAR notas[i]
        suma ← suma + notas[i]
        SI notas[i] >= 6 ENTONCES
            aprobados ← aprobados + 1
        FIN SI
    FIN PARA

    promedio ← suma / 5     // Cálculo matemático; en Java evitar división entera
    MOSTRAR promedio
    MOSTRAR aprobados

    LEER numeroDeNota
    SI numeroDeNota >= 1 Y numeroDeNota <= 5 ENTONCES
        MOSTRAR notas[numeroDeNota - 1]
    SINO
        MOSTRAR "Número de nota inválido"
    FIN SI
FIN
```

Java completo

Java

```
import java.util.Scanner;

public class Main {

    public static void main(String[] args) {

        Scanner teclado = new Scanner(System.in);
        int[] notas = new int[5];
        int suma = 0;
        int aprobados = 0;

        // Guardamos las cinco calificaciones.
        for (int i = 0; i < notas.length; i++) {
            System.out.print("Ingrese nota " + (i + 1) + ": ");
            notas[i] = teclado.nextInt();
        }

        // Consultamos y procesamos los valores guardados.
        System.out.println("Notas registradas:");
        for (int i = 0; i < notas.length; i++) {
            System.out.println((i + 1) + ". " + notas[i]);
```

```
            suma += notas[i];

            if (notas[i] >= 6) {
                aprobados++;
            }
        }

        double promedio = suma * 1.0 / notas.length;
        System.out.println("Promedio: " + promedio);
        System.out.println("Aprobados: " + aprobados);

        System.out.print("¿Qué número de nota desea consultar? ");
        int numeroDeNota = teclado.nextInt();

        if (numeroDeNota >= 1 && numeroDeNota <= notas.length) {
            System.out.println("Valor: " + notas[numeroDeNota - 1]);
        } else {
            System.out.println("Número de nota inválido");
        }

        teclado.close();
    }
}
```

12.3. ¿Por qué numeroDeNota - 1?

El promedio también exige elegir bien el tipo

En el pseudocódigo escribimos `promedio ← suma / 5` para describir la operación matemática. Pero en Java, con dos operandos enteros, `suma / 5` haría una **división entera** antes de asignarla. Por eso el programa utiliza:

Java

```
double promedio = suma * 1.0 / notas.length;
```

Prueba de comprensión: si la suma fuera 11, ¿qué produciría `double promedio = 11 / 5;` y qué produciría `11 / 5.0`? El primero almacena 2.0; el segundo 2.2.

La persona solicita «nota 3» porque cuenta desde uno. Pero el tercer elemento del arreglo está en el índice 2.

Por eso calculamos:

`numeroDeNota - 1`

Cuando el usuario solicita la nota 3, consultamos `notas[2]`.

Primero validamos que el número solicitado esté entre 1 y 5; de lo contrario podríamos intentar un acceso inexistente.

12.4. Versión B: ampliación por etapas (validación y búsqueda)

No es necesario incorporar las dos mejoras a la vez. Sigue este orden:

Etapa B1 — Validación: conserva el programa A y cambia solamente la lectura: si una nota está fuera de 1–10, vuelve a pedir esa misma posición. Comprueba primero que el resto del programa continúa funcionando.

Etapa B2 — Búsqueda: cuando la validación ya funcione, agrega una consulta por **valor**, diferente de la consulta por número de orden. Debes comunicar la primera coincidencia o indicar que el valor no aparece.

La versión A es la **evidencia esencial**. Las etapas B1 y B2 son extensiones progresivas; no constituyen requisitos simultáneos para demostrar los aprendizajes mínimos.

Cuando la versión esencial funciona, podemos agregar dos requisitos:

Validación de entrada: si se introduce una nota fuera de 1–10, volver a solicitar esa misma posición mediante un `do...while`.

Búsqueda: después de mostrar el promedio, solicitar una nota concreta y comprobar mediante búsqueda lineal si aparece. Mostrar el índice de la primera coincidencia o «No encontrada».

No es obligatorio implementar la versión B para demostrar el aprendizaje fundamental. Permite profundizar en ciclos anidados, banderas y búsquedas.

Fragmento de validación para colocar dentro del for

Java

```
int nota;
do {
    System.out.print("Ingrese nota " + (i + 1) + ": ");
    nota = teclado.nextInt();

    if (nota < 1 || nota > 10) {
        System.out.println("Nota inválida");
    }
} while (nota < 1 || nota > 10);

notas[i] = nota;
```

Fragmento de búsqueda para colocar después de registrar los datos

Java

```
System.out.print("Nota que desea buscar: ");
int buscado = teclado.nextInt();

int i = 0;
boolean encontrado = false;

while (i < notas.length && !encontrado) {
    if (notas[i] == buscado) {
```

```
            encontrado = true;
        } else {
            i++;
        }
    }

    if (encontrado) {
        System.out.println("Primera coincidencia en la nota " + (i + 1));
    } else {
        System.out.println("No encontrada");
    }
```

Estos fragmentos **no son programas independientes**; se incorporan al programa completo de la versión A, sustituyendo la lectura simple y agregando la búsqueda después de completar el registro. Si se prefiere, pueden resolverse en dos prácticas separadas.

12.5. Casos de prueba

Para las notas `7, 5, 8, 6, 9`:

Prueba	Resultado esperado
Mostrar el registro	7, 5, 8, 6, 9
Calcular la suma	35
Calcular promedio	7.0
Contar aprobados	4
Consultar nota número 3	8
Consultar nota número 1	7
Consultar nota número 5	9
Consultar nota número 0	Número de nota inválido
Consultar nota número 6	Número de nota inválido
Buscar 8 (versión B)	Primera coincidencia en nota 3
Buscar 4 (versión B)	No encontrada

Para comprobar la versión B, prueba también una nota inválida como 15 durante la entrada: el programa debe volver a pedir la misma posición, no avanzar a la siguiente.

12.6. Qué explicar en la defensa

Un programa puede funcionar y, sin embargo, haber sido construido por copia. Para demostrar comprensión, cada estudiante debe poder explicar:

- Por qué creamos un arreglo y no cinco variables independientes.
- Qué representa `i` durante el recorrido.
- Por qué la condición es `i < notas.length`.
- Qué hace `notas[i] = teclado.nextInt()`.
- Qué función cumplen `suma` y `aprobados`.
- Por qué consultamos `numeroDeNota - 1`.
- Qué sucede cuando se solicita una posición inexistente.

12.7. La siguiente limitación

Con este programa ya podemos almacenar y consultar varias calificaciones. Pero seguimos concentrando nuestras instrucciones en un único `main`. Cuando los programas crezcan, necesitaremos organizar tareas como «calcular promedio» o «buscar una nota» en partes reutilizables.

Esa necesidad nos conducirá a la **Unidad 6: métodos en Java**. Primero debemos comprender cómo manipular correctamente los datos.

---

13. Errores frecuentes y depuración

Para revisarlos con autonomía, podemos agruparlos en tres familias:

Familia	Qué debemos comprobar
Índices y límites	Posición ordinal frente a índice, negativos, último índice y arreglo vacío
Datos y recorridos	Valores realmente registrados, acumuladores, máximos y búsquedas
Cadenas	`length` frente a `length()`, `charAt`, inmutabilidad y `equals()`

En todos los casos, primero realiza una predicción y después ejecuta el programa para comprobarla.

Los arreglos no son difíciles solamente por su sintaxis. Requieren relacionar índices, valores, tamaño y recorrido. Aprender a detectar los siguientes errores es parte del aprendizaje.

13.1. Confundir índice y posición ordinal

Si necesitamos la tercera nota de:

Java

```
int[] notas = {7, 5, 8, 6, 9};
```

Debemos consultar `notas[2]`, no `notas[3]`. La primera posición tiene índice cero.

13.2. Utilizar <= length

Incorrecto para recorrer todas las posiciones:

Java

```
for (int i = 0; i <= notas.length; i++) {
    System.out.println(notas[i]);
}
```

Cuando `i` llega a `notas.length`, se intenta acceder a un elemento inexistente. La condición correcta es `i < notas.length`.

13.3. Usar un índice negativo

`notas[-1]` no representa «no encontrado» ni «el último elemento». Es un acceso inválido. Podemos utilizar -1 como valor de una variable `posicion`, pero **no** como índice para consultar el arreglo.

13.4. Suponer que new int[5] significa cinco notas ingresadas

El arreglo tiene cinco posiciones inicializadas automáticamente en cero. Eso no significa que un estudiante haya recibido cinco calificaciones iguales a cero. Debemos distinguir capacidad y datos registrados.

13.5. Recorrer la capacidad cuando solo parte está ocupada

Si creamos diez posiciones y guardamos cuatro notas, recorrer todas puede incorporar ceros que nunca fueron ingresados. Necesitamos conocer y respetar `cantidad`.

13.6. Reiniciar un acumulador dentro del recorrido

Si escribimos `int suma = 0;` dentro del cuerpo del `for`, perdemos la suma anterior en cada iteración. Debe inicializarse antes.

13.7. Confundir asignación y acumulación

`total = notas[i]` reemplaza el valor anterior. `total = total + notas[i]` incorpora el dato nuevo al resultado acumulado.

13.8. Suponer que consultar copia permanentemente todos los datos

`int tercera = notas[2];` copia un valor entero a `tercera`. Si después modificamos `notas[2]`, `tercera` no se actualiza automáticamente.

13.9. Comenzar máximo en cero para cualquier conjunto

Si todos los números son negativos, un máximo inicial de cero podría producir un resultado incorrecto. Para arreglos no vacíos conviene inicializarlo con `numeros[0]`.

13.10. No comprobar el arreglo vacío

Un arreglo puede tener longitud cero (`new int[0]`). En ese caso no existe `numeros[0]` y tampoco corresponde calcular un promedio dividiendo entre su longitud. Debemos comprobar si hay elementos.

13.11. Buscar sin definir qué significa encontrar

Si un valor aparece tres veces, debemos saber si queremos encontrarlo por primera vez, contar coincidencias o mostrar todas las posiciones. Son problemas distintos.

13.12. Confundir length y length()

- En arreglos: `notas.length`.
- En `String`: `palabra.length()`.

La diferencia es parte de la sintaxis de Java.

13.13. Intentar modificar un String mediante corchetes

`palabra[0] = 'L';` no es una forma válida de cambiar una cadena Java. `String` es inmutable. Sí podemos consultar con `charAt()` u obtener otra cadena mediante sus métodos.

13.14. Comparar cadenas con ==

Para comparar contenidos de `String`, utilizamos `equals()` o `equalsIgnoreCase()` cuando corresponde. `==` no comprueba de forma general la igualdad de contenido.

13.15. Depurar con un trazado

Ampliación: copiar un elemento y compartir un arreglo son cosas diferentes

Compara estos dos fragmentos **independientes**.

Copiar un elemento entero:

```java
int[] a = {3, 5, 8};
int copia = a[0];
a[0] = 10;
System.out.println(copia);  // 3
```

Asignar la referencia del arreglo:

```java
int[] a = {3, 5, 8};
int[] b = a;
b[0] = 10;
System.out.println(a[0]);  // 10
```

En el segundo caso, `a` y `b` hacen referencia **al mismo arreglo**: no se crearon dos conjuntos independientes de elementos. No necesitaremos copiar arreglos completos en los ejercicios esenciales, pero esta diferencia nos prepara para comprender las referencias y los objetos.

Si un programa falla o muestra un resultado incorrecto, registra:

1. Longitud o cantidad utilizada.
2. Índice antes de cada consulta.
3. Valor de `arreglo[i]`.
4. Condición que se evalúa.
5. Cambio del acumulador, contador o bandera.
6. Momento en que termina el recorrido.

No cambies todas las instrucciones a la vez. Localiza primero el paso que produce el error.

---

14. Preparación para la evaluación: leer, predecir, corregir y construir

Cuatro mini-ciclos con productos cerrables

Las actividades A–S forman un **banco**, no una lista que todos deban terminar en una sola clase ni un parcial extenso. Una organización posible, con mini-ciclos de 2–4 clases, es:

Mini-ciclo	Actividades de apoyo	Producto evaluable
1. Guardar, consultar y modificar	A–G, M	Registro fijo y explicación de índices
2. Recorrer y procesar	H–I, N	Suma, promedio y contador con trazado
3. Buscar y comparar	J–K, O–P	Búsqueda inicial y máximo justificado
4. Cadenas e integración	L, Q–S	Consulta de texto y registro consultable

En cada cierre pedimos pseudocódigo breve, código o trazado, dos o tres pruebas y una explicación de aproximadamente un minuto por estudiante. Una pareja entrega un producto y cambia conductor/navegante cada 10–15 minutos.

Las actividades se organizan por complejidad. No es necesario realizar todas en una sola sesión ni utilizarlas como un parcial extenso. Las primeras permiten comprobar comprensión; las siguientes exigen trazados; las últimas requieren pseudocódigo, Java y pruebas.

Nivel 1. Reconocer y comprender

Actividad A. Índices y valores

Dado:

Java

```
int[] datos = {4, 9, 2, 7};
```

Indica la longitud, el primer índice, el último índice y los valores de `datos[0]`, `datos[2]` y `datos[3]`. Explica por qué `datos[4]` no es válido.

Actividad B. Declarar e inicializar

Explica la diferencia entre:

Java

```
int[] edades = new int[4];
```

Y:

Java

```
int[] edades = {15, 16, 15, 17};
```

¿Qué valores tiene cada arreglo inmediatamente después de crearlo?

Actividad C. Longitud y posición

Completa los espacios:

- El primer índice de un arreglo es ____.
- En un arreglo de 8 elementos, el último índice es ____.
- La cantidad de elementos se consulta mediante ____.
- Para obtener el último elemento utilizamos `datos[____]`.

Actividad D. Elegir una estructura

Indica qué información necesitas conservar para: calcular un promedio, volver a mostrar la tercera nota, contar aprobados y buscar un valor. ¿En qué casos alcanza con contador o acumulador y cuándo resulta útil guardar todas las notas en un arreglo?

Nivel 2. Interpretar, predecir y corregir

Actividad E. Trazar un recorrido

Java

```
int[] datos = {3, 5, 7};

for (int i = 0; i < datos.length; i++) {
    System.out.println(datos[i]);
}
```

Predice cada salida. Completa una tabla de `i`, condición y dato consultado. ¿Cuántas iteraciones y cuántas comprobaciones de condición hay?

Actividad F. Encontrar el error de límite

Java

```
int[] notas = {6, 8, 9};

for (int i = 0; i <= notas.length; i++) {
    System.out.println(notas[i]);
}
```

Explica qué ocurre cuando `i` vale 3. Corrige una sola condición y justifica el cambio.

Actividad G. Distinguir copia y actualización

Java

```
int[] numeros = {2, 4, 6};
int copia = numeros[1];
numeros[1] = 10;

System.out.println(copia);
System.out.println(numeros[1]);
```

Predice las dos salidas y explica por qué no son iguales.

Actividad H. Trazar una suma

Java

```
int[] datos = {4, 2, 5};
int suma = 0;
for (int i = 0; i < datos.length; i++) {
    suma += datos[i];
}
System.out.println(suma);
```

Completa una tabla con índice, dato, suma anterior y suma nueva. Predice el resultado.

Actividad I. Contar según un criterio

Para `int[] notas = {6, 4, 7, 9, 5};`, realiza el trazado de un contador de aprobados (nota >= 6). ¿Cuántas veces aumenta?

Actividad J. Buscar un valor

En `{7, 5, 8, 6, 9}`, traza la búsqueda de 8 y después de 4. Indica qué índices se revisan y cómo termina cada búsqueda.

Actividad K. Máximo y mínimo

Para `{-5, -2, -9, -1}`, predice el mayor y el menor. Explica por qué iniciar el mayor en cero produciría un error.

Actividad L. Comprender String

Con `String palabra = "Java";`, predice `palabra.length()`, `palabra.charAt(1)`, `palabra.substring(1,3)` y `palabra.indexOf("va")`. Explica por qué no podemos escribir `palabra[0] = 'L';`.

Nivel 3. Construir, comprobar y justificar

Actividad M. Registrar y mostrar

Construye un programa que solicite cinco edades enteras y las guarde en un arreglo. Después debe mostrarlas en el mismo orden. Escribe primero pseudocódigo y prueba con `15,16,15,17,16`.

Actividad N. Suma y promedio

Construye un programa que almacene cinco números enteros, calcule la suma y el promedio decimal. Prueba con `1,2,3,4,5` (suma 15, promedio 3.0), `2,2,2,2,3` (suma 11, promedio 2.2) y `0,0,0,0,0` (suma 0, promedio 0.0).

Actividad O. Buscar y explicar

Construye un programa que guarde cinco números, solicite otro y muestre si aparece. Ampliación: indicar el índice de la primera coincidencia. Prueba un valor al comienzo, otro al final y otro que no aparezca.

Actividad P. Mayor y menor

Escribe un programa que encuentre el mayor y menor de un arreglo de enteros. Debe funcionar también con todos los valores negativos. Realiza trazados con `{3,8,1,6}`, `{-4,-9,-2,-7}` y `{5}`.

Actividad Q. Contar caracteres

Escribe un programa que solicite una palabra y cuente cuántas veces aparece la letra `a` minúscula. Utiliza `String`, `length()` y `charAt()`. Prueba con `banana` (3), `Java` (2) y `sol` (0). Explica por qué esta versión distingue `a` de `A`.

Actividad R. Registro consultable

Resuelve la versión A del problema integrador: guardar cinco notas, mostrarlas, calcular promedio, contar aprobados y consultar una mediante su número ordinal. Debes validar la consulta. Comprueba los casos de la sección 12.

Actividad S. Extensión con validación y búsqueda

Agrega a la actividad anterior una lectura que repita una nota inválida y una búsqueda lineal de la primera coincidencia. Explica qué ciclo controla la cantidad de posiciones y cuál controla la corrección de cada entrada.

Checkpoint individual de comprensión

Tres preguntas de salida sin computadora

1. Con `int[] notas = {7, 5, 8, 6, 9};`, ¿qué muestra `System.out.println(notas[3]);`? Explica la diferencia entre índice y posición ordinal.
2. Con el mismo arreglo y `suma = 0`, ¿qué valor resulta de recorrer `i = 0` hasta `i < notas.length` y ejecutar `suma += notas[i]`? Traza dos iteraciones.
3. ¿Qué falla al escribir `i <= notas.length` si el cuerpo ejecuta `System.out.println(notas[i]);`? Indica el primer índice inválido y corrige el límite.

Estas preguntas comprueban índices, acumulación y límites sin ayuda de la computadora. Pueden funcionar como checkpoint individual para detectar asimetrías en una pareja.

Después de una práctica en parejas, responde individualmente y sin ejecutar:

1. En un arreglo de cinco elementos, ¿por qué el último índice válido es 4?
2. ¿Qué diferencia existe entre `suma += notas[i]` y `suma = notas[i]`?
3. ¿Qué sucede si buscamos una nota que no aparece y el contador alcanza `notas.length`?

Trabajo en parejas

Cuando se trabaje en parejas, una persona será **conductor** (escribe el código) y la otra **navegante** (anticipa, lee y comprueba). Cambiarán los roles cada 10–15 minutos. Se realizará una entrega por pareja y una explicación individual breve por integrante.

Lista de cotejo del trabajo en pareja:

- ¿Intercambiamos los roles?
- ¿El navegante predijo resultados antes de ejecutar?
- ¿Ambos comprendemos el índice y la condición del recorrido?
- ¿Comprobamos al menos tres casos?

Si una persona no puede explicar el recorrido, se realizará un checkpoint individual con un trazado o corrección de código pequeño.

Criterios de logro

Criterio	Logrado	En proceso	Inicial
Representa datos	Crea y utiliza el arreglo adecuado	Necesita alguna corrección de índice o tipo	No distingue datos individuales y arreglo
Recorre sin errores	Usa límites y actualizaciones correctas	Presenta errores de límite corregibles	No controla el índice
Procesa y busca	Suma, cuenta o busca conforme al problema	Resuelve parcialmente	No conserva el estado necesario
Comprueba	Predice salidas y prueba límites	Pruebas incompletas	Solo ejecuta una vez
Explica	Justifica índices, recorridos y condiciones	Explicación parcial	No puede reconstruir el algoritmo

Autoevaluación: ¿Puedo explicar por qué el primer índice es 0? (Sí / Todavía no). ¿Probé un valor que no aparece en mi búsqueda? (Sí / No). Reflexión breve: «El error que tuve que corregir fue… y lo detecté porque…».

Lo que debemos recordar

En esta unidad aprendimos que procesar datos y conservarlos son tareas diferentes. Un acumulador permite guardar una suma y un contador permite registrar eventos, pero no nos devuelven automáticamente cada valor original. Cuando necesitamos consultar, corregir, buscar o volver a recorrer varios valores del mismo tipo, podemos utilizar un **arreglo**.

Aprendimos a declararlo, crearlo y asignarle elementos. Comprendimos que los índices comienzan en cero, que `length` expresa la cantidad de elementos del arreglo y que el último índice válido es `length - 1` cuando el arreglo no está vacío. Utilizamos `for` y `while` para recorrer las posiciones sin escribir una instrucción diferente para cada dato.

También distinguimos la **capacidad del arreglo** de la **cantidad de posiciones utilizadas**, y recuperamos acumuladores y contadores para obtener resultados a partir de los valores guardados. Después construimos una búsqueda lineal y resolvimos problemas de máximo, mínimo y conteo de coincidencias.

Finalmente, estudiamos operaciones básicas con `String`. Comprobamos que una cadena tiene posiciones y puede recorrerse, pero **no debe confundirse con un arreglo modificable de caracteres**. Utilizamos `length()`, `charAt()`, `equals()`, `indexOf()` y `substring()` según las necesidades de cada problema.

La idea fundamental es esta: **guardar datos en una estructura tiene sentido cuando después necesitamos trabajar con ellos**. Un programa correcto no solamente obtiene un resultado; conserva la información necesaria, respeta los límites y permite comprobar sus operaciones.

En la siguiente unidad aprenderemos a organizar estas tareas en **métodos**. Así podremos separar responsabilidades como registrar, mostrar, sumar o buscar, sin escribir todo el algoritmo dentro de un único `main`.

Próxima clase: plantear cómo evitar repetir la lógica de «mostrar todas las notas» en varios lugares del programa; será el puente hacia la Unidad 6, donde construiremos nuestros primeros métodos.

Evidencia para el próximo corte: un registro de tamaño fijo que permita guardar, mostrar y procesar datos, acompañado de pseudocódigo, al menos tres pruebas y una explicación individual del recorrido.